

ANN_testing

July 14, 2026

1 Evaluation Graphs for ANN

```
[1]: from pathlib import Path
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import display

ROOT = Path.cwd().resolve()
while not (ROOT / 'pyproject.toml').exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.event_detection import build_detector, collect_default_data_paths,
↳ split_train_test
from src.event_detection_pipeline.model_io import load_detector, save_detector
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    auc as calc_auc,
    precision_recall_curve,
    average_precision_score,
)

[2]: data_dir = ROOT / 'src' / 'data'
paths = collect_default_data_paths(data_dir)
train_paths, test_paths = split_train_test(paths)

print(f'Loaded {len(paths)} SCADA files')
print(f'Train files: {len(train_paths)} | Test files: {len(test_paths)}')
```

Loaded 20 SCADA files

Train files: 16 | Test files: 4

```
[3]: # confusion matrix for event detection
history = 48
# history 48 = 1 day, history 96 = 2 days
epochs = 60
batch_size = 128
learning_rate = 3e-4
dropout = 0.0
weight_decay = 1e-4
early_stopping_patience = 8
max_validation_false_alarm_rate = 0.2
# Held-out test: BA=0.6868, recall=0.5609, FAR=0.1872, precision=0.1851

candidate_hidden_sizes = [
    #(32,),
    (64, 32), # best validation configuration
    #(128, 64),
    #(128, 64, 32),
]
```

```
[4]: # RUN only if you want to overwrite the current detector performance results

for hidden_sizes in candidate_hidden_sizes:
    candidate_detector = build_detector(
        train_paths,
        history=history,
        epochs=epochs,
        batch_size=batch_size,
        learning_rate=learning_rate,
        hidden_sizes=hidden_sizes,
        dropout=dropout,
        weight_decay=weight_decay,
        early_stopping_patience=early_stopping_patience,
        max_validation_false_alarm_rate=max_validation_false_alarm_rate,
    )

    save_detector(candidate_detector, directory="../results/
↳supervised_ann_detector")
```

All files saved to /Users/esther/dev/Arsenic-Detection/src/results/supervised_ann_detector/

```
[5]: import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```

from src.event_detection_pipeline.evaluation import (
    plot_roc_curve_event_detection,
    plot_roc_curve_chlorine_prediction,
    plot_chlorine_prediction_quality,
    plot_precision_recall_curve,
)

def plot_metrics(result, title_prefix="", chlorine_indices=None):

    print("Chlorine indices:", chlorine_indices)

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # 1. Confusion Matrix
    cm = confusion_matrix(
        result.event_flags.astype(bool),
        result.alarms.astype(bool),
        labels=[False, True],
        normalize="all",
    )

    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Normal", "Event"],
    )
    disp.plot(
        ax=axes[0, 0],
        colorbar=False,
        values_format=".2f",
    )
    axes[0, 0].set_title(f"{title_prefix}Confusion Matrix")

    # 2. Event detection ROC
    plot_roc_curve_event_detection(
        result,
        title=f"{title_prefix}Event Detection ROC",
        ax=axes[0, 1],
    )

    # 3. True chlorine regression quality (ROC would measure event residuals)
    plot_chlorine_prediction_quality(
        result,
        chlorine_indices=chlorine_indices,
        title=f"{title_prefix}Chlorine Prediction Quality",
        ax=axes[1, 0],
    )

```

```

# 4. Precision-Recall curve
plot_precision_recall_curve(
    result,
    title=f"{title_prefix}Precision-Recall Curve",
    ax=axes[1, 1],
)

plt.tight_layout()
plt.show()
return fig, axes

```

```

[6]: all_detector_summaries = []
# Use candidate_hidden_sizes to load the saved detectors and run tests on them.
for hidden_sizes in candidate_hidden_sizes:
    hidden_sizes_label = "-".join(map(str, hidden_sizes))
    detector = load_detector(
        directory="./results/supervised_ann_detector",
        device="mps",
    )

    selected_test_paths = test_paths
    results = [detector.detect(path) for path in selected_test_paths]

    print(f"Results for detector with hidden sizes: {hidden_sizes_label}")

    # Keep the four held-out test sets separated.
    summary_df = pd.DataFrame(
        [result.summary() for result in results],
        index=[path.name for path in selected_test_paths],
    )
    display(summary_df)

    # Aggregate metrics across the three test sets.
    aggregated_summary = pd.DataFrame(
        [
            summary_df.mean(numeric_only=True),
            summary_df.std(numeric_only=True),
        ],
        index=["mean", "std"],
    )
    display(aggregated_summary)

    detector_summary = summary_df.mean(numeric_only=True).to_dict()
    detector_summary["hidden_sizes"] = hidden_sizes_label
    all_detector_summaries.append(detector_summary)

```

```

    # Plot each test set separately, because each DetectionResult has its own
    ↪ timeline.
    for i, result in enumerate(results):
        plot_metrics(
            result,
            title_prefix=f"Layers: {hidden_sizes_label} | test {i+1} | ",
            chlorine_indices=list(range(result.actual.shape[1]))
        )

# Compare all detectors by their mean metrics.
all_detector_summaries_df = pd.DataFrame(all_detector_summaries).
    ↪ set_index("hidden_sizes")
display(all_detector_summaries_df)

```

Detector successfully loaded from /Users/esther/dev/Arsenic-Detection/src/results/supervised_ann_detector/
Results for detector with hidden sizes: 64-32

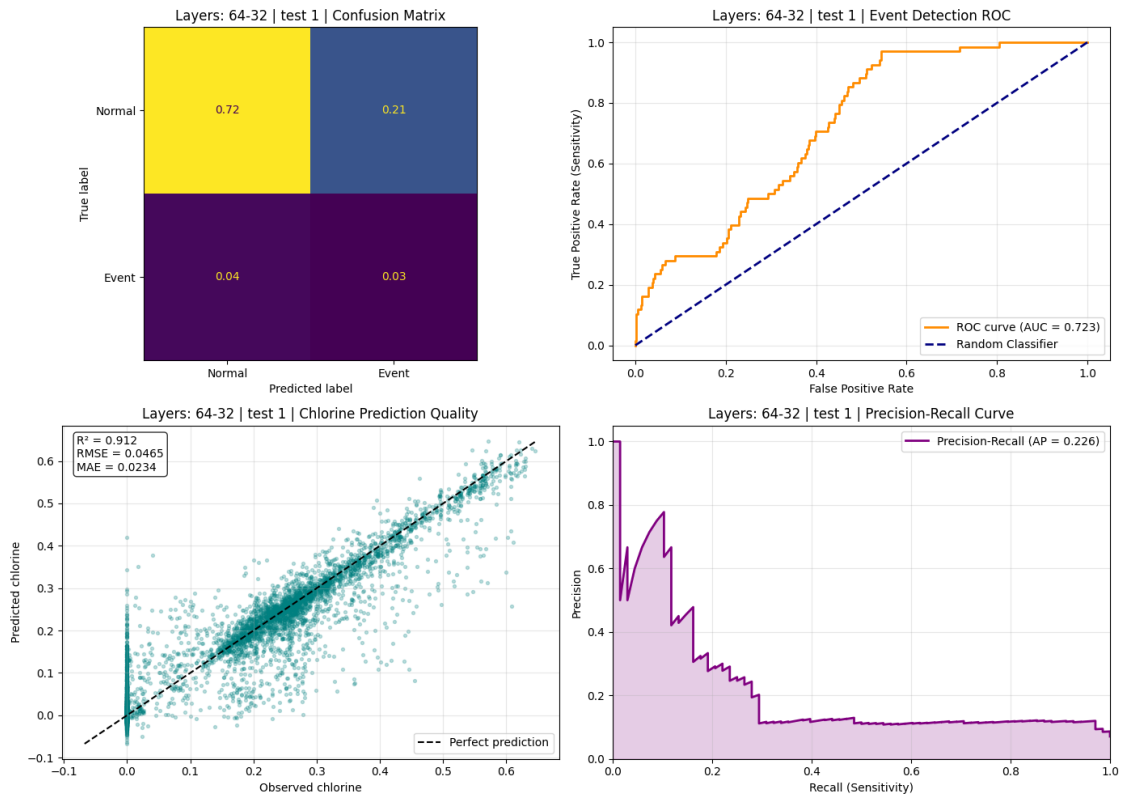
	precision	accuracy	probability_of_detection	recall	\
scada_data_17.npz	0.119469	0.750260	0.397059	0.397059	
scada_data_18.npz	0.127193	0.752341	0.426471	0.426471	
scada_data_19.npz	0.291667	0.876171	0.507246	0.507246	
scada_data_20.npz	0.246964	0.801249	0.924242	0.924242	

	false_alarm_rate	detection_latency_seconds	alarms
scada_data_17.npz	0.222844	0.0	226
scada_data_18.npz	0.222844	9000.0	228
scada_data_19.npz	0.095291	0.0	120
scada_data_20.npz	0.207821	0.0	247

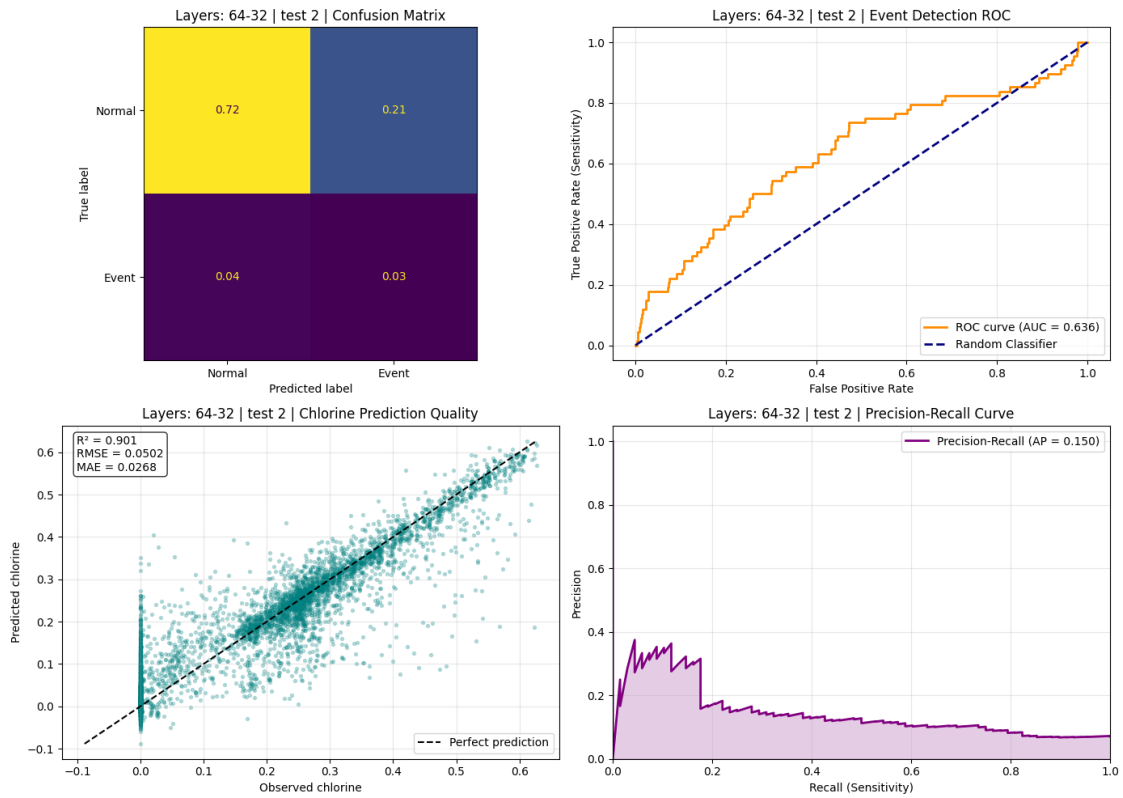
	precision	accuracy	probability_of_detection	recall	\
mean	0.196323	0.795005	0.563755	0.563755	
std	0.086295	0.059017	0.244799	0.244799	

	false_alarm_rate	detection_latency_seconds	alarms
mean	0.18720	2250.0	205.250000
std	0.06168	4500.0	57.615825

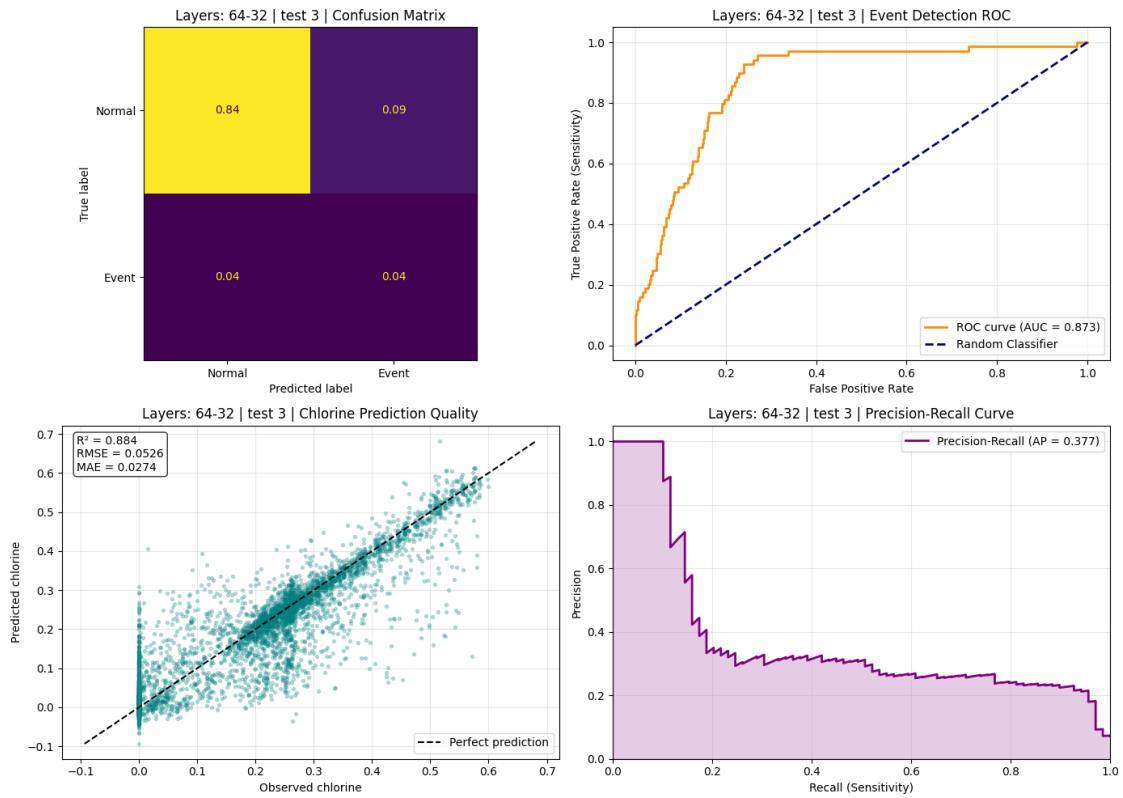
Chlorine indices: [0, 1, 2, 3, 4, 5, 6, 7, 8]



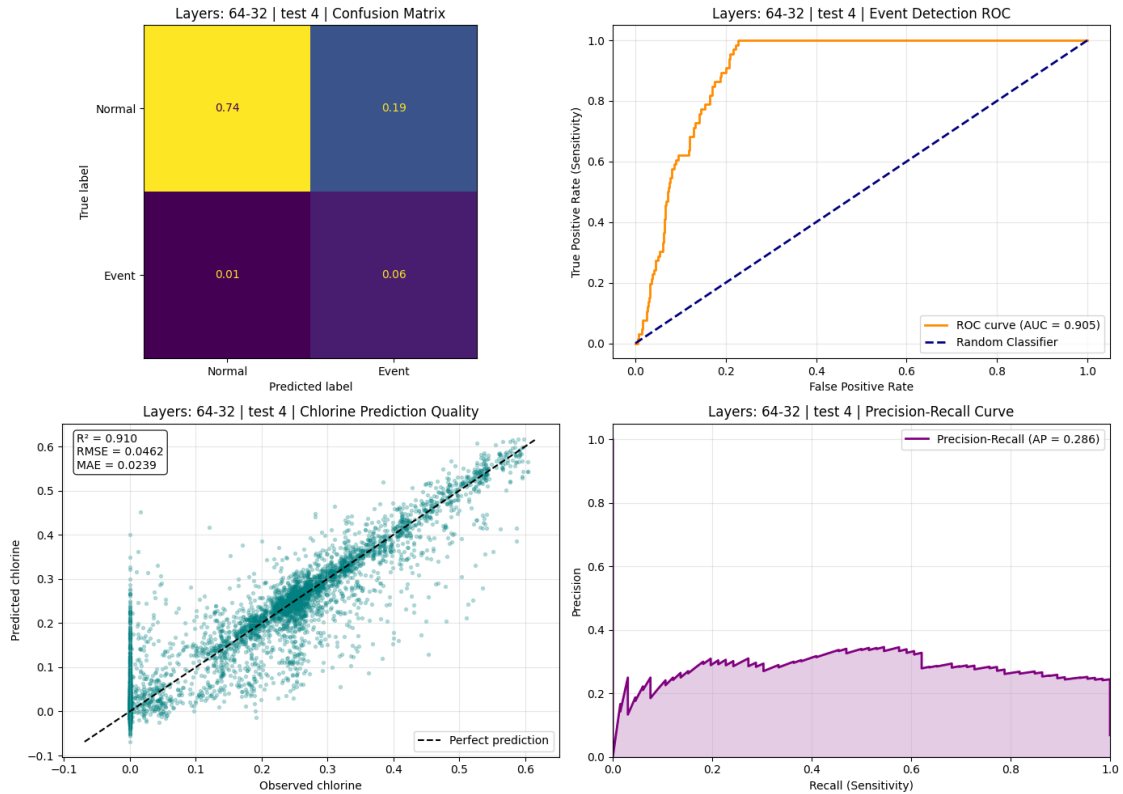
Chlorine indices: [0, 1, 2, 3, 4, 5, 6, 7, 8]



Chlorine indices: [0, 1, 2, 3, 4, 5, 6, 7, 8]



Chlorine indices: [0, 1, 2, 3, 4, 5, 6, 7, 8]



	precision	accuracy	probability_of_detection	recall \
hidden_sizes				
64-32	0.196323	0.795005	0.563755	0.563755

	false_alarm_rate	detection_latency_seconds	alarms
hidden_sizes			
64-32	0.1872	2250.0	205.25

```
[7]: import random

# Plot the tuned 64-32 detector with selected chlorine sensor combinations
detector = load_detector(
    directory="./results/supervised_ann_detector",
    device="mps",
)

#generate 3 random combinations of 3 chlorine output channels
# chlorine_sensor_count = len(detector.target_scalers)
# chlorine_indices_combinations = [random.sample(range(chlorine_sensor_count), 3) for _ in range(3)]

# for chlorine_indices in chlorine_indices_combinations:
```

```

#     selected_test_paths = test_paths[:3]
#     results = [detector.detect(path) for path in selected_test_paths]

#     print(f"Results for detector with hidden sizes: {hidden_sizes_label}")

#     # Keep the three test sets separated.
#     summary_df = pd.DataFrame(
#         [result.summary() for result in results],
#         index=[path.name for path in selected_test_paths],
#     )
#     display(summary_df)

#     # Aggregate metrics across the three test sets.
#     aggregated_summary = pd.DataFrame(
#         [
#             summary_df.mean(numeric_only=True),
#             summary_df.std(numeric_only=True),
#         ],
#         index=["mean", "std"],
#     )
#     display(aggregated_summary)

#     detector_summary = summary_df.mean(numeric_only=True).to_dict()
#     detector_summary["hidden_sizes"] = hidden_sizes_label
#     all_detector_summaries.append(detector_summary)

#     # Plot each test set separately, because each DetectionResult has its own
#     ↪ timeline.
#     for i, result in enumerate(results):
#         plot_metrics(
#             result,
#             title_prefix=f"Layers: {hidden_sizes_label} | test {i+1} | ",
#             chlorine_indices=chlorine_indices
#         )

```

Detector successfully loaded from /Users/esther/dev/Arsenic-Detection/src/results/supervised_ann_detector/